

Practical 4

Aim:

Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce.

Problem Definition:

Scenario: A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends.

As a Data Analytics Engineer, your responsibility is to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce, verify the correctness of the results, and analyze the yearly maximum temperature trends for the organization.

Objectives:

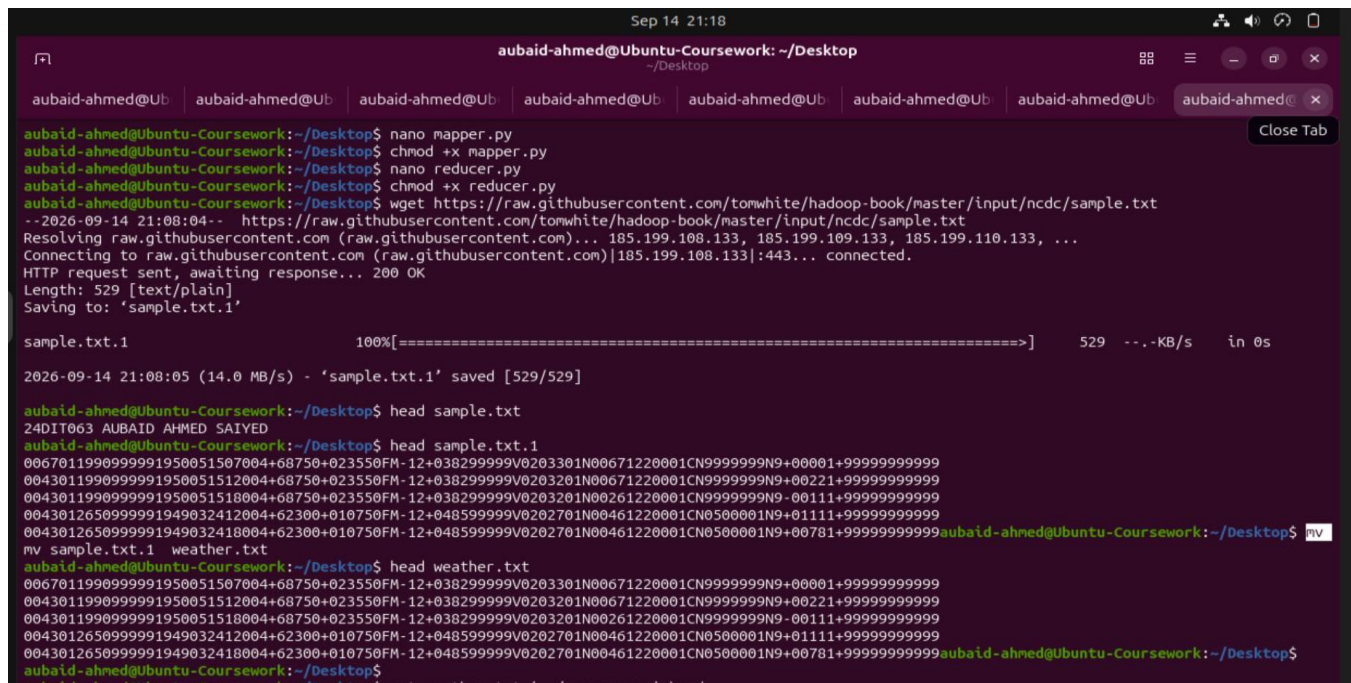
To upload the NCDC weather dataset to HDFS, develop a Mapper program to extract the year and temperature values from each record, develop a Reducer program to determine the maximum temperature recorded for each year, execute the MapReduce job on Hadoop, verify the output, and analyze the yearly maximum temperature trends to identify the hottest year in the dataset.

Observation / Output (Screenshots or command outputs)

1. Upload the NCDC weather dataset to HDFS and examine its structure.

```
hdfs dfs -mkdir -p /user/hadoop/ncdc_input
hdfs dfs -put ncdc_weather_data.txt /user/hadoop/ncdc_input
hdfs dfs -cat /user/hadoop/ncdc_input/ncdc_weather_data.txt | head
```

Output:



```
Sep 14 21:18
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop
aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub: aubaid-ahmed@Ub:
Close Tab

aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ wget https://raw.githubusercontent.com/tomwhite/hadoop-book/master/input/ncdc/sample.txt
--2026-09-14 21:08:04-- https://raw.githubusercontent.com/tomwhite/hadoop-book/master/input/ncdc/sample.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 529 [text/plain]
Saving to: 'sample.txt.1'

sample.txt.1                                100%[=====]                529  --.-KB/s   in 0s

2026-09-14 21:08:05 (14.0 MB/s) - 'sample.txt.1' saved [529/529]

aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ head sample.txt
24DIT063 AUBAID AHMED SAIIYED
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ head sample.txt.1
0067011990999991950051507004+68750+023550FM-12+038299999V0203301N00671220001CN9999999N9+00001+9999999999
0043011990999991950051512004+68750+023550FM-12+038299999V0203201N00671220001CN9999999N9+00221+9999999999
0043011990999991950051518004+68750+023550FM-12+038299999V0203201N00261220001CN9999999N9-00111+9999999999
0043012650999991949032412004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+01111+9999999999
0043012650999991949032418004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+00781+9999999999
mv sample.txt.1 weather.txt
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ head weather.txt
0067011990999991950051507004+68750+023550FM-12+038299999V0203301N00671220001CN9999999N9+00001+9999999999
0043011990999991950051512004+68750+023550FM-12+038299999V0203201N00671220001CN9999999N9+00221+9999999999
0043011990999991950051518004+68750+023550FM-12+038299999V0203201N00261220001CN9999999N9-00111+9999999999
0043012650999991949032412004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+01111+9999999999
0043012650999991949032418004+62300+010750FM-12+048599999V0202701N00461220001CN0500001N9+00781+9999999999
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | head
```

2. Develop the Mapper program to extract year and temperature values.

```
import sys

for line in sys.stdin:
    line = line.strip()

    if not line:
        continue

    year = line[15:19]
    temp = line[87:92]

    if temp != "+9999" and temp != "-9999":
        print(year, temp)
```

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | ./mapper.py | head
1950 +0000
1950 +0022
1950 -0011
1949 +0111
1949 +0078
```

3. Develop the Reducer program to determine the maximum temperature per year.

```
GNU nano 8.7.1
#!/usr/bin/env python3

import sys

current_year = None
max_temp = None

for line in sys.stdin:
    year, temp = line.strip().split()

    temp = int(temp)

    if current_year == year:
        if temp > max_temp:
            max_temp = temp
    else:
        if current_year is not None:
            print(current_year, max_temp)

        current_year = year
        max_temp = temp

if current_year is not None:
    print(current_year, max_temp)
```

Output:

```
Sep 14 21:22
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x mapper.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x reducer.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ wget https://raw.githubusercontent.com/tomwhite/hadoop-book/master/input/ncdc/sample.txt
--2026-09-14 21:08:04-- https://raw.githubusercontent.com/tomwhite/hadoop-book/master/input/ncdc/sample.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 529 [text/plain]
Saving to: 'sample.txt.1'

sample.txt.1          100%[=====] 529 --.-KB/s  in 0s

2026-09-14 21:08:05 (14.0 MB/s) - 'sample.txt.1' saved [529/529]
```

4. Compile the program, create the JAR, and execute the MapReduce job on Hadoop.

hadoop jar maxtemp.jar MaxTempDriver /user/hadoop/ncdc_input /user/hadoop/ncdc_output

Output:

```
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | ./mapper.py | head
1950 +0000
1950 +0022
1950 -0011
1949 +0111
1949 +0078
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | ./mapper.py | sort | ./reducer.py
1949 111
1950 22
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ start-dfs.sh
start-yarn.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Ubuntu-Coursework]
Starting resourcemanager
Starting nodemanagers
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -mkdir -p /ncdc/input
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -put weather.txt /ncdc/input/
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -ls /ncdc/input
Found 1 items
-rw-r--r-- 2 aubaid-ahmed supergroup 529 2026-09-14 21:12 /ncdc/input/weather.txt

aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-input /ncdc/input/weather.txt \
-output /ncdc/output \
-mapper mapper.py \
-reducer reducer.py \
-file mapper.py \
-file reducer.py
2026-09-14 21:12:50,519 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar5097137799335871410/] [] /tmp/streamjob303996652486434897.jar tmpDir=null
2026-09-14 21:12:54,078 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-14 21:12:54,932 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-14 21:12:56,290 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aubaid-ahmed/.staging/job_1789
400462536_0001
2026-09-14 21:12:57,904 INFO mapred.FileInputFormat: Total input files to process : 1
2026-09-14 21:12:58,435 INFO mapreduce.JobSubmitter: number of splits:2
2026-09-14 21:12:59,946 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1789400462536_0001
2026-09-14 21:12:59,947 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-14 21:13:00,959 INFO conf.Configuration: resource-types.xml not found
2026-09-14 21:13:00,959 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-14 21:13:02,410 INFO impl.YarnClientImpl: Submitted application application_1789400462536_0001
2026-09-14 21:13:02,515 INFO mapreduce.Job: The url to track the job: http://Ubuntu-Coursework:8088/proxy/application_1789400462536_0001/
2026-09-14 21:13:02,530 INFO mapreduce.Job: Running job: job_1789400462536_0001
2026-09-14 21:13:30,514 INFO mapreduce.Job: Job job_1789400462536_0001 running in uber mode : false
2026-09-14 21:13:30,522 INFO mapreduce.Job: map 0% reduce 0%
2026-09-14 21:13:48,644 INFO mapreduce.Job: map 50% reduce 0%
2026-09-14 21:13:50,030 INFO mapreduce.Job: map 100% reduce 0%
2026-09-14 21:14:05,870 INFO mapreduce.Job: map 100% reduce 100%
```

```
Sep 14 21:24
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop

Reduce output records=2
Spilled Records=10
Shuffled Maps =2
Failed Shuffles=0
Merged Map outputs=2
GC time elapsed (ms)=525
CPU time spent (ms)=9840
Physical memory (bytes) snapshot=787836928
Virtual memory (bytes) snapshot=8189628416
Total committed heap usage (bytes)=443547648
Peak Map Physical memory (bytes)=296669184
Peak Map Virtual memory (bytes)=2722033664
Peak Reduce Physical memory (bytes)=217337856
Peak Reduce Virtual memory (bytes)=2745880576

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
Bytes Read=794
File Output Format Counters
Bytes Written=19

2026-09-14 21:14:09,461 INFO streaming.StreamJob: Output directory: /ncdc/output
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -ls /ncdc/output
Found 2 items
-rw-r--r-- 2 aubaid-ahmed supergroup 0 2026-09-14 21:14 /ncdc/output/_SUCCESS
-rw-r--r-- 2 aubaid-ahmed supergroup 19 2026-09-14 21:14 /ncdc/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000
1949 111
1950 22
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
1949 111
```

Verify the output generated in HDFS.

hdfs dfs -ls /user/hadoop/ncdc_output

hdfs dfs -cat /user/hadoop/ncdc_output/part-r-00000

Output:

```
Sep 14 21:25
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop

Peak Reduce Virtual memory (bytes)=2745880576

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
Bytes Read=794
File Output Format Counters
Bytes Written=19

2026-09-14 21:14:09,461 INFO streaming.StreamJob: Output directory: /ncdc/output
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -ls /ncdc/output
Found 2 items
-rw-r--r-- 2 aubaid-ahmed supergroup 0 2026-09-14 21:14 /ncdc/output/_SUCCESS
-rw-r--r-- 2 aubaid-ahmed supergroup 19 2026-09-14 21:14 /ncdc/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000
1949 111
1950 22
```

5. Analyze the yearly maximum temperature trends and identify the hottest year.

```
hdfs dfs -cat /user/hadoop/ncdc_output/part-r-00000 | sort -k2 -n -r | head -1
```

Output:

```

Sep 14 21:25
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop
Peak Reduce Virtual memory (bytes)=2745880576
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=794
File Output Format Counters
Bytes Written=19
2026-09-14 21:14:09,461 INFO streaming.StreamJob: Output directory: /ncdc/output
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -ls /ncdc/output
Found 2 items
-rw-r--r-- 2 aubaid-ahmed supergroup 0 2026-09-14 21:14 /ncdc/output/_SUCCESS
-rw-r--r-- 2 aubaid-ahmed supergroup 19 2026-09-14 21:14 /ncdc/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000
1949 111
1950 22
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
1949 111
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
1949 111
```

Result:

The NCDC weather dataset was successfully uploaded to HDFS, and a MapReduce job was developed and executed to determine the maximum temperature recorded for each year. The Mapper extracted the year and temperature fields from each weather record, and the Reducer aggregated the values to compute the yearly maximum temperature. The output was verified in HDFS, and the yearly maximum temperature trends were analyzed to identify the hottest year in the dataset, demonstrating the effectiveness of MapReduce for large-scale climate data analysis.

GitHub: [BDA-SHP-2026/Practical4 at main · AubaidAhmedSaiyed/BDA-SHP-2026](https://github.com/AubaidAhmedSaiyed/BDA-SHP-2026)

Key Questions / Analysis:

Q1. How is weather data parsed and processed using MapReduce?

The raw NCDC weather records are stored as fixed-width text lines, with each field (station ID, date, year, temperature, quality code, etc.) occupying a specific character position within the line. In the Map phase, each line of the input file is passed to the Mapper as a value, and the Mapper parses the line using substring operations to extract the year and the air temperature field. It also checks the quality code to filter out corrupted or missing readings (represented as 9999). Valid records are then emitted as intermediate key-value pairs. This parsing approach allows Hadoop to process each record independently and in parallel across the cluster, making it possible to handle very large weather datasets efficiently.

Q2. What key-value pairs are generated by the Mapper?

For every valid weather record, the Mapper emits the year as the key (a Text object, e.g., "1949") and the corresponding air temperature as the value (an IntWritable object, e.g., 111, representing 11.1°C). So a single record such as a reading of 22°C recorded in 1950 would produce the pair (1950, 22). Hadoop's shuffle and sort phase then groups all temperature values sharing the same year key together, so that each Reducer receives one year along with an iterable list of all temperature readings recorded for that year.

Q3. How does the reducer identify the maximum temperature for a given year?

For each year (key) received, the Reducer iterates through the list of temperature values associated with that key and keeps track of the largest value seen so far, initializing a variable to Integer.MIN_VALUE and updating it with Math.max() as it processes each temperature reading. Once all the values for that year have been examined, the Reducer emits the year along with the final maximum value as the output key-value pair. Because this comparison is done independently for each year's group of values, the Reducer effectively produces one maximum-temperature record per year.

Q4. What challenges arise when processing large climate datasets?

Large climate datasets such as the NCDC archive pose several challenges: the raw data is often in a fixed-width or inconsistent text format that requires careful parsing, and it can contain missing, corrupted, or sentinel values (like 9999) that must be filtered using quality codes. The sheer volume of records (spanning decades and thousands of stations) makes single-machine processing impractical, requiring distributed storage and parallel computation. Data skew can also occur, since some years or stations may have far more records than others, leading to uneven load across Reducers. Additionally, ensuring data quality, handling schema variations across different file formats, and managing storage and I/O overhead for very large files add further complexity to the analysis pipeline.

Q5. How does Hadoop improve scalability and performance for weather data analytics?

Hadoop improves scalability by distributing the NCDC dataset across multiple DataNodes in HDFS and processing it in parallel using the MapReduce framework, so that adding more nodes to the cluster directly increases both storage and processing capacity. The Map phase processes data locally on the node where it is stored (data locality), reducing network overhead, while the shuffle and sort phase efficiently groups related records for the Reduce phase. Hadoop also provides fault tolerance through data replication and automatic re-execution of failed tasks, ensuring that long-running climate analysis jobs complete reliably even on commodity hardware. This makes it possible to process decades of weather data and compute yearly statistics far faster than traditional single-machine approaches.

Supplementary Problems:**1. Find the minimum temperature recorded for each year.**

Answer)

- Modify the Reducer logic to initialize the tracking variable to Integer.MAX_VALUE instead of Integer.MIN_VALUE.
- Use Math.min() instead of Math.max() while iterating through the temperature values for each year.
- Recompile the program, re-run the MapReduce job, and store the results in a new output directory.
- Verify the output to confirm that the minimum temperature per year has been correctly computed.

```
Sep 14 21:36
aubaid-ahmed@Ubuntu-Coursework: ~/Desktop
Peak Reduce Virtual memory (bytes)=2745880576
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=794
File Output Format Counters
Bytes Written=19
2026-09-14 21:14:09,461 INFO streaming.StreamJob: Output directory: /ncdc/output
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -ls /ncdc/output
Found 2 items
-rw-r--r-- 2 aubaid-ahmed supergroup 0 2026-09-14 21:14 /ncdc/output/_SUCCESS
-rw-r--r-- 2 aubaid-ahmed supergroup 19 2026-09-14 21:14 /ncdc/output/part-00000
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000
1949 111
1950 22
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
1949 111
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
1949 111
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano supp.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano reducer_min.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | ./mapper.py | sort | ./reducer_min.py
bash: ./reducer_min.py: Permission denied
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x reducer_min.py
chmod: cannot access 'reducer_min.py': No such file or directory
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ chmod +x reducer_min.py
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ cat weather.txt | ./mapper.py | sort | ./reducer_min.py
1949 78
1950 -11
aubaid-ahmed@Ubuntu-Coursework:~/Desktop$ nano mapper.py
```

2. Calculate average yearly temperatures.

Answer)

- In the Reducer, accumulate the sum of all temperature values and count the number of readings for each year.
- After iterating through all values for a key, compute the average as sum divided by count.
- Emit the year along with the computed average temperature as the output.
- Run the job and verify that the average yearly temperatures are written correctly to HDFS.

3. Identify the year with the highest recorded temperature overall.

Answer)

- Run the yearly maximum temperature MapReduce job to generate one maximum value per year.
- Read the output file and sort the year-temperature pairs in descending order of temperature.
- Select the record with the single highest temperature value as the overall hottest year.
- Cross-verify the result using the hdfs dfs -cat and sort commands on the output file.

Tools / Technology used:

Hadoop 3.x, HDFS, Java JDK 8/11, Eclipse / IntelliJ IDEA, Linux/Ubuntu Environment.

Conclusion:

Through this practical, the NCDC weather dataset was uploaded to HDFS, and a MapReduce program consisting of a Mapper and a Reducer was developed to determine the maximum temperature recorded for each year. The Mapper parsed each weather record to extract the year and temperature fields, while the Reducer aggregated these values to compute the yearly maximum. The MapReduce job was executed successfully on Hadoop, and the output was verified in HDFS. The yearly maximum temperature trends were analyzed to identify the hottest year, demonstrating how MapReduce enables efficient, scalable, and fault-tolerant processing of large-scale climate datasets compared to traditional single-machine approaches.